Step 2 — Every Run Saved to a Database and Findable by ID — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: after Step 1, a run's result lives only in the one HTTP reply it was sent in — once that response is gone, there's no record it ever happened. This step saves every run to SQLite (Doc12's pattern): a `"started"` row is written **before** the team even runs, then updated to `"completed"` or `"failed"` after — so even a run that crashes mid-way leaves an honest, queryable record instead of silently vanishing. `GET /runs/{id}` reads one back by id, and a request ID ties each request's scattered log lines together into one traceable story. **If not:** "what happened on that run yesterday?" has no answer at all.

After this step the project runs: every `POST /run-task` saves a row, and `GET /runs/1` shows it. Files not shown here don't change.

The shared change

Changed from Step 1: the reply now carries the saved run's ID and status.

```
# src/api/schemas.py
# The API's request/reply shapes. TaskResponse now also carries the saved
# run's id and status.
from pydantic import BaseModel, Field

class TaskRequest(BaseModel):
    topic: str = Field(min_length=1, max_length=200)

class TaskResponse(BaseModel):
    run_id: int          # new in Step 2
    status: str          # new in Step 2
    accepted: bool
    reason: str
    article: str
```

Back to this step · Hint 1 · Hint 2 · Solution

Solution — a `db/` package, a checked run shape, and a request ID

Story — `db/database.py`: this puts every SQL statement behind a small, named function — Doc12's `runs_db.py` and Build Task, unchanged except for `DB_PATH`. Each write wraps its own `with conn:` block, so the `"started"` row is committed on its own, immediately, rather than waiting on the whole request to finish. `?` placeholders (never string-formatted SQL) keep a topic like `x'); DROP TABLE runs; --` safely treated as plain text, never as SQL syntax. (Also create an empty `db/__init__.py`.) **If not:** SQL would be scattered through the routes, and a crashed run could vanish or stay stuck at `"started"` forever.

```
# src/db/database.py
# All SQL behind small named functions. A run is saved "started" first,
# then updated to "completed" or "failed"; ? placeholders keep input safe.
import sqlite3
from datetime import datetime, timezone

CREATE_RUNS_TABLE = """
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        log TEXT,
        status TEXT,
        created_at TEXT
    )
"""

def get_connection():
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute(CREATE_RUNS_TABLE)
    return conn

def insert_started_run(conn, topic):
    now = datetime.now(timezone.utc).isoformat()
    with conn:
        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
            (topic, "started", now),
        )
```

```
    return cursor.lastrowid

def finish_run(conn, run_id, output, log_text):
    with conn:
        conn.execute(
            "UPDATE runs SET output = ?, log = ?, status = ? WHERE id = ?",
            (output, log_text, "completed", run_id),
        )

def fail_run(conn, run_id):
    with conn:
        conn.execute(
            "UPDATE runs SET status = ? WHERE id = ?",
            ("failed", run_id),
        )

def fetch_run(conn, run_id):
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    if row is None:
        return None
    return dict(row)
```

Story — `db/models.py`: this defines one saved run's exact public shape as a Pydantic model, used as `GET /runs/{id}`'s `response_model` (Doc12's `RunRecord` pattern) — FastAPI filters the response to only these fields, regardless of what's actually in the database row. **If not:** a column added to the database later — say, an internal cost figure — would leak straight through to every client.

```
# src/db/models.py
# Run: one saved run's shape, checked on the way out — a column added to the
# table later never leaks to the client by accident.
from pydantic import BaseModel

class Run(BaseModel):
    id: int
    input: str
    output: str | None     # None while the run is "started" or "failed"
    log: str | None
    status: str
    created_at: str
```

Changed from Step 1: `routes.py` opens the connection, saves the run around the team's work, puts the request ID on every log line, and adds `GET /runs/{run_id}` with a 404.

```
# src/api/routes.py
# The API routes. POST /run-task now saves a "started" run before the team
# runs and "completed"/"failed" after; GET /runs/{run_id} reads one back or
# returns 404; every log line carries the request id.
from fastapi import APIRouter, HTTPException

from src.api.schemas import TaskResponse
from src.db.database import get_connection, insert_started_run     # new
from src.db.database import finish_run, fail_run, fetch_run        # new
from src.db.models import Run                                      # new
from src.pipeline import run_pipeline
from src.utils.logger import get_logger

router = APIRouter()
conn = get_connection()   # new in Step 2: one connection for the app

INTERNAL_ERROR = "Internal error. Please try again later."

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/run-task", response_model=TaskResponse)
# changed in Step 2: + http_request, to read the request id
def run_task(request, http_request):
    logger = get_logger(__name__)
    request_id = http_request.state.request_id   # new in Step 2
    if not request.topic.strip():
        raise HTTPException(status_code=400, detail="topic cannot be empty")

    # new in Step 2: saved as "started" BEFORE the team runs
    run_id = insert_started_run(conn, request.topic)
    logger.info(f"[{request_id}] run {run_id} started")
    try:
        outcome = run_pipeline(request.topic)
    except Exception:
        fail_run(conn, run_id)   # new in Step 2
        logger.exception(f"[{request_id}] run {run_id} failed")
        raise HTTPException(status_code=500, detail=INTERNAL_ERROR)

    # new in Step 2: the routing steps become the log column
    log_text = "\n".join(outcome["steps"])
    finish_run(conn, run_id, outcome["article"], log_text)
    logger.info(f"[{request_id}] run {run_id} completed: {outcome['reason']}")
    return TaskResponse(
        run_id=run_id,
        status="completed",
```

```
        accepted=outcome["accepted"],
        reason=outcome["reason"],
        article=outcome["article"],
    )

# new in Step 2: look one run up by id
@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id):
    run = fetch_run(conn, run_id)
    if run is None:
        raise HTTPException(status_code=404, detail="run not found")
    return run
```

Changed from Step 1: `main.py` adds Doc12's request-ID middleware.

```
# src/api/main.py
# The app uvicorn starts, now with a middleware that puts a short request id
# on every request and returns it in the X-Request-ID header.
import uuid                              # new in Step 2

from fastapi import FastAPI

from src.api.routes import router
from src.config import load_config

load_config()

app = FastAPI(title="ContentForge Pro")
app.include_router(router)

# new in Step 2: one short id per request, set once for every route
@app.middleware("http")
async def add_request_id(request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response
```

Changed from Step 1: the same tests, with the same names, now also check the saved rows — a valid run is saved as `"completed"`, a bad body writes nothing, a crash is saved as `"failed"`, a wrong ID is a 404, and every reply has an `X-Request-ID`.

```
# tests/api/test_routes.py
# The API's own behavior — status codes and replies — with Project 4's
```

```
# team FAKED: no model calls, no cost, same every time.
# Run from the repo root: pytest tests/api -v
# The app still refuses to start without OPENAI_API_KEY — any value
# works here, because no real model is ever called.
from fastapi.testclient import TestClient

from src.api import routes
from src.api.main import app

client = TestClient(app)

def fake_run_pipeline(topic):
    return {
        "article": "E-Bikes Explained\n\nUp to 100 km. [battery-review]",
        "accepted": True,
        "reason": "reviewer accepted the draft",
        "steps": [
            "research_agent: no research yet",
            "END: reviewer accepted the draft",
        ],
    }

def broken_run_pipeline(topic):
    raise RuntimeError("simulated internal failure")

def use_fake_team(monkeypatch, fake):
    monkeypatch.setattr(routes, "run_pipeline", fake)

# new in Step 2: count saved rows
def row_count():
    sql = "SELECT COUNT(*) FROM runs"
    return routes.conn.execute(sql).fetchone()[0]

def failed_count():
    sql = "SELECT COUNT(*) FROM runs WHERE status = ?"
    return routes.conn.execute(sql, ("failed",)).fetchone()[0]

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_valid_request_is_answered(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 200
    body = response.json()
```

```
    assert body["status"] == "completed"
    assert body["accepted"] is True
    assert "[battery-review]" in body["article"]

    # new in Step 2: read the saved run back
    saved = client.get("/runs/" + str(body["run_id"])).json()
    assert saved["input"] == "electric bikes"
    assert saved["status"] == "completed"
    assert "END: reviewer accepted the draft" in saved["log"]

def test_bad_body_is_422(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    before = row_count()   # new in Step 2
    response = client.post("/run-task", json={})
    assert response.status_code == 422
    assert row_count() == before

def test_blank_topic_is_400(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "   "})
    assert response.status_code == 400
    assert response.json() == {"detail": "topic cannot be empty"}

def test_internal_error_is_a_plain_500(monkeypatch):
    use_fake_team(monkeypatch, broken_run_pipeline)
    before = failed_count()   # new in Step 2
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 500
    expected = {"detail": "Internal error. Please try again later."}
    assert response.json() == expected
    assert failed_count() == before + 1

# new in Step 2
def test_unknown_run_is_404():
    response = client.get("/runs/999999")
    assert response.status_code == 404

# new in Step 2
def test_every_reply_has_a_request_id():
    response = client.get("/health")
    assert len(response.headers["X-Request-ID"]) == 8
```

Run it (from the repo root):

```
pytest tests/api -v
uvicorn src.api.main:app --reload
```

```
tests/api/test_routes.py::test_health PASSED
tests/api/test_routes.py::test_valid_request_is_answered PASSED
tests/api/test_routes.py::test_bad_body_is_422 PASSED
tests/api/test_routes.py::test_blank_topic_is_400 PASSED
tests/api/test_routes.py::test_internal_error_is_a_plain_500 PASSED
tests/api/test_routes.py::test_unknown_run_is_404 PASSED
tests/api/test_routes.py::test_every_reply_has_a_request_id PASSED
```

Then `curl localhost:8000/runs/1` (shown wrapped to fit the page — really one line):

```
{"id":1,"input":"electric bikes","output":"E-Bikes: How Far ...",
 "log":"research_agent: no research yet\nanalysis_agent: research done,
 no findings yet\n...\nEND: reviewer accepted the draft",
 "status":"completed","created_at":"2026-01-01T00:00:00.000000+00:00"}
```

Server log for one request (every line has the same ID):

```
2026-01-01 00:00:00,000 src.api.routes INFO [9ee100c1] run 1 started
2026-01-01 00:00:09,000 src.api.routes INFO [9ee100c1] run 1 completed:
reviewer accepted the draft
```

(The second line is really one line — shown wrapped.) The tests write to the same `runs.db` as the app; it's in `.gitignore`, and you can delete it any time.